

PROJECT REPORT: IoT Surveillance System with Firebase Cloud

Subject: Automation, Serial Communication (UART) and Cloud Computing

Technology: Arduino Uno, ESP32, Firebase (Google)

Realized by: Jaafar Marfouq / Amine Mokid / Rayane Mahtaj

1. Introduction

1.1 Context and Motivation

In Industry 4.0, uploading information to the Cloud is essential for long-term data analysis. However, many industrial systems rely on robust controllers (like the Arduino) that manage sensors but lack native connectivity. This project aims to create a bridge between the physical world (Arduino) and the digital world (Firebase Cloud) for effective thermal monitoring.

1.2 Project Objective

This project aims to create a complete system capable of:

1. **Managing Locally (Edge Computing):** The Arduino measures the temperature, displays the data on an LCD, and activates the cooling (fan) and the alarm (buzzer) if necessary.
2. **Transmitting:** The Arduino sends the temperature data in real-time via a serial link (UART).
3. **Connecting (Gateway):** The ESP32 receives this data and sends it to a Google Firebase real-time database for remote monitoring.

1.3 System Architecture

The system is divided into two distinct functional blocks:

- **The Control Block (Arduino Uno):** It is autonomous. Even if the Wi-Fi cuts out, the Arduino continues to cool the system and trigger local alerts. It manages all the core "business" logic.
- **The Communication Block (ESP32):** It acts as a transparent gateway. It listens to the Arduino and repeats everything it hears to Firebase.

2. Hardware List and Roles

The components are mainly centered around the Arduino.

1. **Arduino Uno (Local Central Unit)**
 - **Role:** Main brain. It reads the sensor, drives the relay/fan, manages the LCD screen, and the Buzzer. It formats the data for transmission.
 - **Voltage:** 5V

2. ESP32 (Wi-Fi Gateway)

- **Role:** Network interface. It makes no decisions. It receives data on its Serial port (Rx) and pushes it to Firebase.
- **Voltage:** 3.3V

3. Temperature Sensor (DHT11)

- **Connection:** Plugged directly into the Arduino.

4. 5V Relay & DC Fan

- **Connection:** The relay is controlled by the Arduino (Pin 7 / Pin 8) to close the 5V circuit of the fan.

5. I2C LCD Screen & Buzzer

- **Connection:** Plugged into the Arduino for the local human-machine interface.

3. Architecture and Wiring

The hardware architecture has been adapted to match the defined software logic.

3.1 Peripheral Management (Arduino Side)

The Arduino provides the necessary 5V power supply to the Relay, the LCD, and the Sensor.

3.2 Serial Communication (The UART Bridge)

Communication between the two boards is done via the Serial protocol (Tx/Rx) at a speed of 9600 bauds.

- **Arduino Tx (Pin 1) -> ESP32 Rx (GPIO 16)**
- **Arduino Rx (Pin 0) -> ESP32 Tx (GPIO 17)**
- **Common GND:** Essential for the voltage reference.

3.3 Global Connection Diagram (Updated according to the Code)

1. Arduino Zone:

- DHT11: Analog Pin A1
- Relay (Fan): Digital Pin 8
- Buzzer: Digital Pin 7
- I2C LCD: SDA (A4) / SCL (A5) Pins
- Tx: To ESP32 Rx.

2. ESP32 Zone:

- Rx: GPIO 16 (Receiving Arduino data)
- Tx: GPIO 17
- Wi-Fi: Connected to the "inwi Home" network.

4. Logic and Software Implementation

System operation is governed by a distributed state machine between the two microcontrollers.

4.1 Operating Algorithm

A. Slave Logic (Arduino Uno):

The Arduino executes a monitoring loop every 3 seconds:

1. **Acquisition:** Reads humidity and temperature values via the DHT11 sensor.
2. **Cooling Decision:**
 - IF Temperature $\geq 21^{\circ}\text{C}$: Activate Relay (Fan starts).
 - ELSE: Deactivate Relay.
3. **Local Alert Decision:**
 - IF Temperature $> 25^{\circ}\text{C}$ (Critical threshold): Activate Buzzer and display "URGENT" on the LCD.
4. **Transmission:** Send data to the ESP32 via the serial port in CSV format: Temperature, Humidity, FanState.

B. Master Logic (ESP32):

The ESP32 is permanently listening:

1. **Reception:** Waits for a line of data on the UART port.
2. **Parsing:** Splits the received string to extract numerical values.
3. **Cloud:** Sends the data to the real-time Firebase database.
4. **Remote Alert (WhatsApp):**
 - IF Temperature $> 25^{\circ}\text{C}$ AND Alert not active: Immediately send a WhatsApp message "CRITICAL Alert".
 - IF Temperature remains $> 25^{\circ}\text{C}$ after 15 seconds: Send a reminder message.
 - IF Temperature drops back to $\leq 25^{\circ}\text{C}$: Send a confirmation message "Return to normal".

4.2 Source Code: Physical Control (Arduino Uno - C++)

```
#include "DHT.h"
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

const int DHT_PIN = A1;
const int DHT_TYPE = DHT11;

const int RELAY_PIN = 8;
const int BUZZER_PIN = 7;    // Buzzer on pin 7
const float TEMP_SEUIL = 21.0; // Threshold for Fan
const float TEMP_CRITIQUE = 25.0; // Critical value for URGENT

// 16x2 I2C LCD (address 0x27)
LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht(DHT_PIN, DHT_TYPE);
```

```

void setup() {
  Serial.begin(9600);
  pinMode(RELAY_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(RELAY_PIN, HIGH);
  digitalWrite(BUZZER_PIN, LOW); // Buzzer off by default

  dht.begin();

  // LCD initialization
  lcd.init();
  lcd.backlight();
}

void loop() {
  delay(3000);

  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  if (isnan(temp) || isnan(hum)) return;

  int fan = (temp >= TEMP_SEUIL) ? 1 : 0;
  digitalWrite(RELAY_PIN, (fan == 1) ? LOW : HIGH);

  // Buzzer Management
  if (temp > TEMP_CRITIQUE) {
    digitalWrite(BUZZER_PIN, HIGH); // buzzer activated
  } else {
    digitalWrite(BUZZER_PIN, LOW); // buzzer deactivated
  }

  // LCD Display
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Temp: ");
  lcd.print(temp, 1);
  lcd.print("C");

  lcd.setCursor(0, 1);
  if (temp > TEMP_CRITIQUE) {

```

```

    lcd.print("URGENT!");
} else {
    lcd.print("      "); // clear line
}

```

```

// Serial Transmission
Serial.print(temp, 2);
Serial.print(",");
Serial.print(hum, 2);
Serial.print(",");
Serial.println(fan);

```

```

delay(3000);
}

```

4.3 Source Code: IoT Gateway (ESP32 - MicroPython)

(Note: Minor OCR scanning errors in the Python script from the PDF have been corrected for readability)

```

import machine
import network
import urequests
import time

```

```

# Global variables for SMS alert management

```

```

LAST_SMS_TIME = 0

```

```

SMS_INTERVAL = 15 # Minimum interval between two alert SMS (in seconds)

```

```

ALERT_ACTIVE = False # Indicates if the alert (temperature > 25°C) is currently active

```

```

def send_sms(message):

```

```

    try:

```

```

        phone = "+0000000000000" # Complete number with area code

```

```

        apikey = "YOUR_API_KEY" # API key received from CallMeBot

```

```

        # Encode spaces and special characters

```

```

        message_encoded = message.replace(" ", "%20")

```

```

        url =

```

```

"[https://api.callmebot.com/whatsapp.php?phone=](https://api.callmebot.com/whatsapp.php?phone=)
&text={}&apikey={}".format(
    phone,

```

```

        message_encoded,
        apikey
    )
    response = urequests.get(url)
    print("SMS sent", response.text)
    response.close()

    # Update the time of the last SMS sent
    global LAST_SMS_TIME
    LAST_SMS_TIME = time.time()

except Exception as e:
    print("SMS sending error:", e)

# WIFI Connection
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
print("Connecting to WiFi...")
wlan.connect('SSID', 'PASSWORD')

for i in range(10):
    if wlan.isconnected():
        break
    time.sleep(1)

if wlan.isconnected():
    print("Connected to WiFi:", wlan.ifconfig())
else:
    print("Unable to connect to WiFi")
    raise SystemExit

uart = machine.UART(1, tx=17, rx=16, baudrate=9600) # TX/RX according to wiring
FIREBASE_URL =
'[https://led-statue-a48fc-default-rtdb.firebaseio.com/](https://led-statue-a48fc-default-rtdb.firebaseio.com/)'

def send_to_firebase(path, value):
    if wlan.isconnected():
        try:
            urequests.put(FIREBASE_URL + path + '.json', json=value).close()
            print(f"Data sent: {path} -> {value}")
        except Exception as e:
            print("Firebase Error:", e)

```

```

else:
    print("No WiFi connection, data not sent")

while True:
    if uart.any():
        line = uart.readline()
        if line:
            line = line.decode().strip()
            print("Received from Arduino:", line) # Display for debug

            try:
                parts = line.split(',')
                if len(parts) == 3: # Expected format "temp, hum, fan"
                    temp = float(parts[0])
                    hum = float(parts[1])
                    fan = int(parts[2])

                    # Send to Firebase
                    send_to_firebase('/Temperature', temp)
                    send_to_firebase('/Humidity', hum)
                    send_to_firebase('/Fan', fan)

                    # WHATSAPP ALERT LOGIC
                    current_time = time.time()

                    if temp > 25:
                        if not ALERT_ACTIVE:
                            send_sms("CRITICAL Alert | Temperature reached {}°C. Immediate action
required.".format(temp))
                            ALERT_ACTIVE = True
                        elif current_time - LAST_SMS_TIME > SMS_INTERVAL:
                            # Reminder message (15s interval)
                            send_sms("REMINDER Alert | Temperature is still at {}°C.".format(temp))
                        elif temp <= 25 and ALERT_ACTIVE:
                            # Temperature drops below or equals 25C and alert was active
                            send_sms("Temperature below critical threshold.")
                            ALERT_ACTIVE = False

            except Exception as e:
                print("UART conversion error:", e)

            time.sleep(0.1)

```

5. Conclusion

This implementation demonstrates the effectiveness of a distributed architecture. The Arduino ensures the immediate safety of the installations (cooling at 21°C, alarm at 25°C) without depending on the internet, while the ESP32 offers a modern supervision layer via Firebase and WhatsApp. This hybrid system successfully combines industrial robustness and IoT connectivity.